

День 1. Бази даних, частина 2

Транзакції та ACID, індексування, моделювання даних, SQL та реляційна алгебра

Транзакції та властивості ACID

Продовжуємо тему баз даних. У першій частині ми розібрали реляційну модель, ключі й нормалізацію. Тепер переходимо до того, як база працює з даними під час змін, як ми ці дані запитуємо і як організовуємо їх на етапі проєктування. Почнемо з транзакцій.

Транзакція — це група операцій, які база виконує як єдине ціле. Класичний приклад — переказ грошей з одного рахунку на інший. Тут дві дії: списати суму з першого рахунку і додати її на другий. Ці дві дії мають або виконатися обидві, або не виконатися жодна. Якщо гроші спишуться з першого рахунку, але через збій не дійдуть до другого, дані стануть неправильними. Саме щоб цього не сталося, існують транзакції, а їхню надійність описують чотири властивості, відомі за аббревіатурою ACID.

Перша властивість — атомарність, англійською atomicity. Вона означає «все або нічого». Транзакція або виконується повністю, або скасовується повністю. Якщо щось пішло не так посередині, усі вже зроблені зміни відкочуються назад, ніби нічого й не було.

Друга — узгодженість, англійською consistency. Вона гарантує, що транзакція переводить базу з одного коректного стану в інший коректний стан. Усі правила й обмеження, які ми задали, наприклад що баланс не може бути від'ємним, лишаються виконаними і до, і після транзакції.

Третя — ізолюваність, англійською *isolation*. Вона означає, що транзакції, які виконуються одночасно, не заважають одна одній. Поки одна транзакція ще не завершилась, інші не бачать її проміжних, незавершених результатів. Зовні все виглядає так, ніби транзакції виконуються по черзі, навіть якщо насправді працюють паралельно.

Четверта — довговічність, англійською *durability*. Вона гарантує, що після успішного завершення транзакції, тобто після підтвердження, її результати збережуться назавжди, навіть якщо одразу після цього вимкнеться живлення чи станеться збій системи.

Запам'ятай ці чотири властивості як набір: атомарність, узгодженість, ізолюваність, довговічність. Питання на ACID — одне з найчастіших у цій темі, і зазвичай просять зіставити назву властивості з її означенням.

Індексування, резервне копіювання та розподіленість

Тепер кілька важливих концепцій систем баз даних. Перша — індексування. Індекс — це додаткова структура, яка прискорює пошук даних. Аналогія проста: це як предметний покажчик у кінці книжки. Замість того щоб гортати всю книжку сторінка за сторінкою в пошуках потрібного слова, ти заглядаєш у покажчик і одразу бачиш номер сторінки. Так само індекс дозволяє базі не переглядати всю таблицю рядок за рядком, а швидко знайти потрібні записи. Але за це є плата: індекс займає додаткову пам'ять і трохи сповільнює вставлення та оновлення даних, бо індекс теж треба оновлювати. Тому індекси створюють на тих стовпцях, за якими часто шукають.

Друга концепція — резервне копіювання та відновлення. Резервна копія, англійською *backup*, — це збережений знімок даних, з якого можна відновити базу після збою, помилки чи втрати. Відновлення

часто спирається на журнал транзакцій, який фіксує всі зміни, тож базу можна повернути в коректний стан.

Третя — розподіленість і реплікація. Розподіленість означає, що дані зберігаються не на одному сервері, а розкидані по кількох вузлах.

Реплікація — це коли ми тримаємо копії одних і тих самих даних на кількох вузлах. Навіщо це потрібно? По-перше, для надійності: якщо один вузол вийде з ладу, дані лишаться на інших. По-друге, для швидкості: запити можна розподіляти між копіями. Платою тут є складність підтримання узгодженості між копіями.

І останнє в цьому блоці — безпека даних. Сюди входить розмежування доступу: кожен користувач має лише ті права, які йому потрібні. Це включає автентифікацію, тобто перевірку, хто ти такий, авторизацію, тобто перевірку, що тобі дозволено, шифрування даних і систему ролей та привілеїв.

Моделювання даних

Тепер про те, як базу проєктують ще до її створення. Цей процес називається моделюванням даних, і він проходить через три рівні моделей: концептуальну, логічну та фізичну. Це важлива тема, бо тут когнітивний рівень вищий — від тебе чекають розуміння й застосування, а не просто впізнавання.

Концептуальна модель — найвищий, найабстрактніший рівень. Вона описує, які сутності існують у предметній області та як вони пов'язані, без жодних технічних деталей. Наприклад: є студенти, є курси, студенти записуються на курси. Жодних типів даних, таблиць чи ключів тут ще немає. Це погляд з висоти пташиного польоту.

Логічна модель — наступний рівень. Тут абстрактні сутності перетворюються на таблиці з атрибутами, визначаються первинні й

зовнішні ключі, дані нормалізуються. Але ця модель ще не прив'язана до конкретної системи керування базами даних.

Фізична модель — найдетальніший рівень. Тут уже все конкретно: типи даних кожного стовпця, індекси, спосіб зберігання, особливості конкретної DBMS, наприклад PostgreSQL чи Oracle. Це готовий план для реалізації.

Запам'ятай напрямок: від концептуальної до фізичної ми рухаємось від абстрактного до конкретного, від «що» до «як саме».

Окремо варто знати ER-модель, тобто модель «сутність-зв'язок», англійською Entity-Relationship. Це найпопулярніший спосіб зображення концептуальної моделі. У ній є сутності, наприклад студент чи курс, є їхні атрибути, наприклад ім'я чи назва, і є зв'язки між сутностями. Зв'язки бувають різної кардинальності: один до одного, один до багатьох і багато до багатьох. Наприклад, один студент може записатися на багато курсів, і один курс може мати багато студентів — це зв'язок багато до багатьох. ER-моделі малюють у різних нотаціях. Найвідоміші — нотація Чена, де сутності зображають прямокутниками, а зв'язки ромбами, і нотація «вороняча лапка», англійською Crow's Foot, де кардинальність показують спеціальними значками на кінцях ліній.

Мова SQL та її підмови

Тепер про те, як ми звертаємось до реляційної бази. Робимо це мовою SQL, що розшифровується як Structured Query Language, тобто структурована мова запитів. SQL поділяється на кілька підмов за призначенням, і їх треба чітко розрізняти. Зауваж: на іспиті не вимагають писати конкретні запити, важливо розуміти, за що відповідає кожна підмова.

Перша підмова — DDL, Data Definition Language, мова визначення даних. Вона відповідає за структуру бази: створення таблиць, зміну їхньої структури, видалення таблиць. Ключові команди — створити, змінити, видалити, англійською `create`, `alter`, `drop`. Запам'ятай: DDL працює зі структурою, зі схемою бази, а не з самими даними.

Друга — DML, Data Manipulation Language, мова маніпулювання даними. Вона працює із самими даними всередині таблиць: додати рядок, оновити, видалити, вибрати дані. Ключові команди — вибрати, вставити, оновити, видалити, англійською `select`, `insert`, `update`, `delete`. Найчастіша операція — вибірка даних, тобто `select`.

Третя — DCL, Data Control Language, мова керування доступом. Вона відповідає за права користувачів: надати доступ і відібрати доступ, англійською `grant` і `revoke`.

Четверта — TCL, Transaction Control Language, мова керування транзакціями. Вона керує транзакціями, про які ми говорили на початку: підтвердити транзакцію, відкотити її, встановити точку збереження, англійською `commit`, `rollback`, `savepoint`.

Підсумуй для себе так: DDL — структура, DML — дані, DCL — доступ, TCL — транзакції. Питання часто просять зіставити команду з її підмовою, тому ці чотири пари варто завчити твердо.

Реляційна алгебра

І завершуємо реляційною алгеброю. Це математична основа запитів: набір операцій над таблицями, результатом кожної з яких знову є таблиця. Розберемо основні операції.

Відбір, англійською `selection`. Він повертає лише ті рядки, що задовольняють задану умову. Наприклад, усі студенти зі стипендією

понад певну суму. Відбір працює з рядками.

Проекція, англійською *projection*. Вона повертає лише вибрані стовпці, а решту відкидає. Наприклад, тільки імена й телефони студентів. Проекція працює зі стовпцями. Дуже важливо не плутати відбір і проекцію: відбір — рядки за умовою, проекція — стовпці. Це класична іспитова пастка.

Об'єднання, англійською *union*. Воно бере два набори рядків з однаковою структурою і повертає всі рядки разом, без дублікатів.

Перетин, англійською *intersection*. Він повертає лише ті рядки, що є в обох наборах одночасно.

Різниця, англійською *difference*. Вона повертає рядки, що є в першому наборі, але відсутні в другому.

Декартів добуток, англійською *cartesian product*. Він з'єднує кожен рядок першої таблиці з кожним рядком другої. Якщо в першій таблиці десять рядків, а в другій п'ять, результат матиме п'ятдесят рядків. Сам по собі він рідко потрібен, але є основою для наступної операції.

З'єднання, англійською *join*. Це, по суті, декартів добуток із подальшим відбором рядків, у яких збігається значення спільного атрибута. Саме *join* дозволяє об'єднувати пов'язані дані з різних таблиць, наприклад додати до кожної оцінки ім'я студента, узявши його з таблиці студентів за спільним ключем. Це найважливіша операція для роботи з кількома таблицями.

І остання — ділення, англійською *division*. Воно повертає ті значення з першої таблиці, які пов'язані з усіма значеннями другої таблиці.

Класичний приклад: знайти студентів, які склали абсолютно всі предмети зі списку. Ділення — найскладніша для розуміння операція, але на іспиті достатньо знати її загальну ідею: «пов'язані з усіма».

Підсумок

Підсумуємо другу частину. Транзакція — це група операцій як єдине ціле, а її надійність описують чотири властивості ACID: атомарність, узгодженість, ізольованість, довговічність. Індекс прискорює пошук ціною додаткової пам'яті. Реплікація — це копії даних на кількох вузлах для надійності й швидкості. Моделювання проходить три рівні: концептуальний, логічний, фізичний, а ER-модель описує сутності та зв'язки між ними. SQL ділиться на чотири підмови: DDL для структури, DML для даних, DCL для доступу, TCL для транзакцій. А реляційна алгебра дає операції над таблицями, серед яких найважливіше не плутати відбір рядків і проєкцію стовпців.

На цьому розділ баз даних завершено. Послухай обидві частини ще раз і переходь до тестів — саме активне пригадування закріплює матеріал найкраще.